Your files, explained

What every file in the digital twin node does, and what it is for

EEE40016 Design of Smart Power Grids
Node: digital twin server (Munir Aslam)
12 October 2026

> A walk through the project folder, `Downloads\EEE40016\project`, one folder at a time: what each file does, why it exists, and how to run it when it is something you run. The models and results are explained in **Digital_Twin_Guide.pdf**, and the Raspberry Pi set-up in **Putting_It_Together.pdf**.

Contents

1. How the folder is organised 3

2. How the files work together 4

3. Which file do I open when... 5

4. The settings (config/) 6

5. The input data (data/) 7

6. The model (src/) 7

7. The checks (tests/) 10

8. The helpers (tools/) 12

9. Teammates' files (team/) 13

10. Results (results/) 14

11. Notes and documents 15

12. Everything else in the project folder 16

1. How the folder is organised

Every file has a home, and the folder names say what kind of thing lives there.

Folder	What is in it
`config/`	the settings: every number the study uses, in two files
`data/`	the inputs: the IEEE street, the cars, the sun and the weather
`src/`	the model itself: the code that simulates an evening
`tests/`	the checks that prove each piece works (V1 to V9 and more)
`tools/`	helpers: running many experiments at once, the Raspberry Pi, the SINCAL cross-check
`team/`	each teammate's starting file
`results/`	everything the runs produced: numbers and figures
`notes/`	written findings: the audit, the 28 design changes, each check's result
`docs/`	the three guides, and the rules teammates' code must follow
`pi_package/`	what goes on the Raspberry Pi
`models/`, `logs/`	SINCAL projects; logs from long runs

If you only open five files: `notes/AUDIT.md` (every plan commitment checked, with the results and what is left), `config/study.json` (all the settings), `src/sweep.py` (one simulated evening, the heart of the model), `src/stages.py` (runs the experiments) and `results/stages/summary.txt` (the main results as numbers).

2. How the files work together

Most of the code exists to run one simulated evening, and most of the rest runs that evening many times or checks it. Figure 1 shows how the pieces connect.

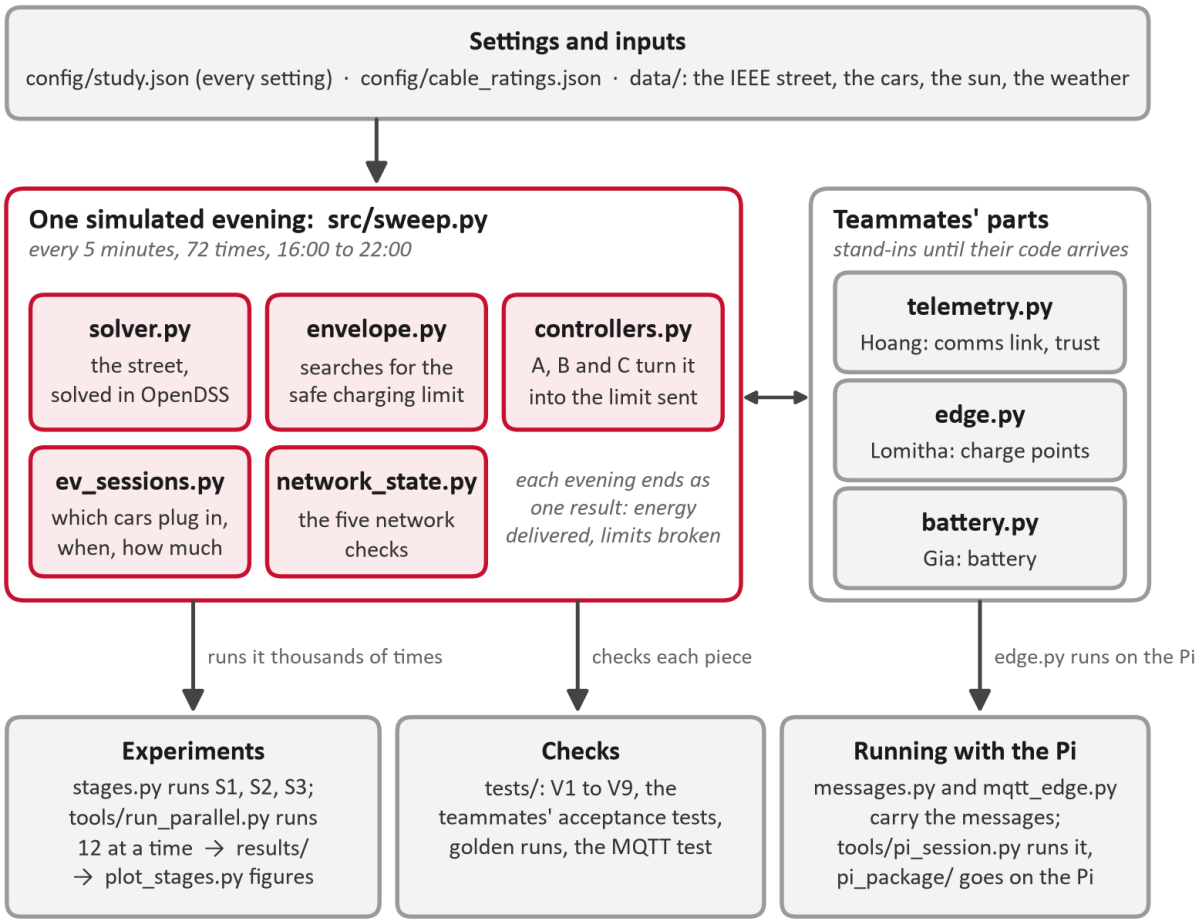


Figure 1. The settings and inputs feed one simulated evening in sweep.py, which uses the files in red and plugs in the three teammates' parts. Experiments run it thousands of times, the tests check each piece, and the Pi path carries the same evening over MQTT.

One evening, file by file

When you run an evening, this is what happens and which file does it:

1. `sweep.py` reads the settings from `config/study.json`, and `feeder_csv.py` and `solver.py` build the street in OpenDSS.
2. `ev_sessions.py` loads the cars for that seed from `data/sessions/`: who plugs in, when, and how much energy they want.
3. Every 5 minutes, the street is set to its true state: house demand, solar, and the battery's response (`battery.py`).
4. The meter readings go through Hoang's comms link (`telemetry.py`), which delays, drops or freezes them and scores how far they can be trusted.

5. `envelope.py` finds the highest safe charging limit from what arrived, asking `network_state.py` whether each try breaks a limit.
6. `controllers.py` turns that into the limit sent (A, B or C), which goes back down the link to the charge points (`edge.py`).
7. The chargers draw power, the true street is solved again, and `network_state.py` records any limit that broke.
8. After 72 steps, `sweep.py` returns one result: the energy delivered, the steps with a violation, the mean trust, and so on.

3. Which file do I open when...

I want to	Open or run
change a setting (the time window, limits, margins, timeouts)	`config/study.json`
see the final results	`results/stages/summary.txt` and `stage_S1.png` (main case); `results/stress_s2/` (stress)
run one evening and watch it step by step	`py src/sweep.py --controller B --latency 60 --loss 0.1 --seed 7 --trace`
redraw the result figures	`py src/plot_stages.py`
check a teammate's code	`py tests/test_node_acceptance.py --team hoang`
run an evening with the Raspberry Pi	`py tools/pi_session.py --pi <user>@<Pi address>`
... with everyone's parts and the battery on the Pi	`py tools/pi_session.py --pi <user>@<Pi address> --team all --battery-on-pi`
hand the whole project to a teammate	`py tools/make_handover.py`
prove a change did not alter any result	`py tests/golden_runs.py --check --flag on`
find out why a design decision was made	`notes/plan_amendments.md`
see what is left to do	`notes/AUDIT.md`, last section
see the rules teammates' code must follow	`docs/INTERFACES.md`

4. The settings (config/)

`config/study.json` SETTINGS

Every number the study depends on, in one file, so nothing is hidden in the code. Change a number here, never in the code, then run the golden runs to see what it moved. Many entries have a `_note` beside them saying where the number came from. The main sections:

Section	What it sets
`cases`	the two network set-ups: `benchmark` (the published IEEE case, 800 kVA, used only by V1; never change it) and `study` (150 kVA transformer, 1.00 pu source, 22 EV charge points, demand case s1, stress case s2)
`network`	the IEEE European LV feeder: 906 buses, 55 houses
`study_window`	16:00 to 22:00, a control step every 5 minutes (72 steps), meter readings every 30 seconds
`constraints`	the five limits: voltage 216.2 to 253 V, cable current, transformer loading 100%, transformer hot spot, voltage unbalance 2%
`controllers`	the names of A, B and C, as in Table 2 of the plan
`envelope`	the safe-limit search: 0 to 7.2 kW per car, to within 0.1 kW, with margins that grow with the age of the data
`controller_settings`	the 1.4 kW fallback, the 15-minute timeouts, the trust formula, setpoint schedules on
`sweep`	the delays (0 to 600 s) and loss levels (0 to 40%) of stage S1
`sincal`	settings for building the SINCAL copy of the street

`config/cable_ratings.json` SETTINGS

The current rating of each cable type, and the transformer's thermal data for the hot-spot model. The IEEE package gives no cable ratings, so these were estimated from the conductor sizes and are marked provisional. Checking them against AS/NZS 3008.1.1 is still on your list, and it matters: cable current is one of the two limits that bind most often.

5. The input data (data/)

`data/feeder/` INPUT

The official IEEE European LV Test Feeder package, unmodified: the line and load tables (`European_LV_CSV/`), the published solutions that V1 compares against (`Solutions/`), and the IEEE's description of the feeder.

`data/sessions/` INPUT

The charging sessions: which car plugs in, when, and how much energy it wants. One file per seed (7 to 19) and demand case, made once and frozen, so every run at a seed sees exactly the same cars. The name tells you which: `seed_7_n22_s1.json` is seed 7, 22 charge points, main case; `_s2` is the stress case. Files without `s1` or `s2`, and the `_standin` ones, belong to earlier scenarios and are kept so old results can be regenerated.

`data/vista/` INPUT

The VISTA 2024-25 travel survey (open data from Transport Victoria), and the arrival times and energy needs worked out from it (`ev_sessions_empirical.csv`, `ev_sessions_summary.json`).

`data/pv_profile.csv` INPUT

Rooftop solar output for every minute of the day, from a clear-sky model for Melbourne with one cloud passing in the afternoon.

`data/ambient.csv, data/weather/` INPUT

The outdoor temperature for every minute, from Bureau of Meteorology observations, for the transformer's hot-spot model. `weather/` holds the raw BoM download it was made from.

`data/scenario_provenance.json` INPUT

Where the solar and temperature data came from and how they were made, so the inputs can be traced.

6. The model (src/)

These are the files that simulate the street. The ones marked **STAND-IN** are placeholders for your teammates' parts; everything else is yours.

The heart of one evening

`sweep.py` CORE

Runs one simulated evening, 16:00 to 22:00, for one controller and one comms condition (a delay and a loss rate), following the steps in Section 2. It returns one result: the energy delivered, the steps that broke a limit, the mean trust and more. It also finds controller A's fixed limit for each seed (`calibrate_static`: the largest fixed limit that never breaks anything all evening). Everything else in `src/` exists to serve this file.

Run: `py src/sweep.py --controller B --latency 60 --loss 0.1 --seed 7`

`envelope.py` CORE

The twin's main calculation: the highest charging power every connected car could draw right now without breaking a limit (the "envelope"). It tries a value, solves the street, asks `network_state.py` whether anything broke, and halves the range each time (a bisection) until it is within 0.1 kW. It also picks which 22 houses have an EV charger. V3 and V4 check it.

`network_state.py` CORE

Turns a solved power flow into a pass or a fail. It checks the five limits (voltage, cable current, transformer loading, transformer hot-spot temperature, voltage unbalance), says which one is closest to breaking, and carries the transformer's temperature from one step to the next (IEC 60076-7).

`solver.py` CORE

The power-flow engine behind one simple interface: set the loads, solve, read the voltages and currents. It runs the street in OpenDSS, which is free and has no size limit. A SINCAL version is also here for cross-checks, but the laptop's SINCAL licence stops at 50 nodes, so the full street can't run in it.

`feeder_csv.py` CORE

Reads the IEEE feeder package into Python and checks it hard before anything uses it: units, bus references, every line and load. Run on its own, it writes the checks to `notes/feeder_validation.md`.

Run: `py src/feeder_csv.py`

`controllers.py` CORE

The three controllers the study compares, named as in Table 2 of the plan. A: one fixed limit all evening, ignoring the data. B: slides between the envelope and 1.4 kW depending on how far the data can be trusted. C: the full envelope while the data is fresh, then straight back to the fixed limit once the 15-minute timeout passes, as the Australian trials do.

The cars

`ev_sessions.py` CORE

Decides which cars plug in, when, and how much energy they want, then freezes the result in `data/sessions/`. It also follows each car through the evening: energy delivered so far, and when it is full. The two demand cases are made here: s1, where a car plugs in on about a third of evenings (the main case), and s2, where it plugs in on 70% (the stress case).

`vista_sessions.py` DATA PREPARATION

Builds the car data from the VISTA travel survey: when each driver last got home, and how far they drove that day, turned into kWh (0.17 kWh per km, 90% charger efficiency, at most 60 kWh). Writes `data/vista/`.

Your teammates' parts

`telemetry.py` STAND-IN FOR HOANG

What the twin actually knows, as opposed to what is true. Meter readings are sent every 30 seconds and get delayed, dropped or frozen on the way, in simulated time, and a trust score falls from 1 to 0 as the data gets older. The downlink is here too: the limits sent back can also arrive late or not at all. Hoang's code replaces it.

`edge.py` STAND-IN FOR LOMITHA

The charge points and their controller. It applies the limit to each of the 22 chargers, picks the right schedule entry when new limits stop arriving, falls back to 1.4 kW when everything is too old, and reports what each car drew. The Pi runs this same class. Lomitha's code replaces it.

`battery.py` STAND-IN FOR GIA

A 100 kWh, 50 kVA battery in the middle of the street (bus 686), with the volt-var and volt-watt response AS/NZS 4777.2 asks for and a state-of-charge model with losses. Gia's code replaces it.

`plugins.py` INTEGRATION

Swaps a teammate's class in by name, so nobody edits anyone else's code. This is what makes `--team hoang` (or `gia`, `lomitha`, `all`) work on the tests, the Pi runs and the experiments.

Talking to the Raspberry Pi

`messages.py` SHARED

The message contract: every message between the laptop and the Pi, with its fields, units and a check, now at version 2, including the two battery messages used when the battery runs on the Pi. The laptop and the Pi must use the identical file, which is why it is copied into the Pi package and nobody re-types it.

`mqtt_edge.py` PI LINK

The MQTT layer, in two pairs. `PiNode` runs on the Pi and wraps the charge-point controller; `MqttEdge` runs in the twin and behaves exactly like the in-process edge, so `sweep.py` cannot tell whether the chargers are simulated or on the Pi. `BatteryNode` and `RemoteBattery` do the same for the battery. Each part ignores messages that are not addressed to it, which the laptop's broker sometimes hands out.

Running the experiments

`stages.py` YOU RUN THIS

Runs the experiments as the plan's Table 9 defines them: S1 (7 delays by 6 loss levels, controllers A, B and C), S2 (data frozen for 0, 2, 5 or 8 minutes), S3 (the battery switched off) and S1H (the six cells where schedules matter most). One seed per run; the results go to `results/stages/`, and `--summarise` turns them into `summary.txt`. S4 is not built yet: it needs decisions from Gia.

Run: `py src/stages.py --stage S1 --seed 7`

Run: `py src/stages.py --summarise`

`timeout_sensitivity.py` EXTRA ANALYSIS

Tests how much the answer depends on the 15-minute timeout, by rerunning with other timeouts on the seven congested seeds. Results and a figure in `results/timeout_sensitivity/`.

Run: `py src/timeout_sensitivity.py --all`

`refine_crossover.py` EARLIER WORK

From the first crossover study in late September: it refined the grid around the break-even point. `stages.py` replaced it, and its results are in `results/superseded/`.

Figures

`plot_stages.py` YOU RUN THIS

Draws the result figures from a results folder: `stage_S1.png` (where B and C beat the fixed limit, across delay and loss), `stage_S2.png` (frozen data) and `stage_S3.png` (battery on against off). It reads `results/stages/` unless you give it another folder.

Run: `py src/plot_stages.py`

Run: `py src/plot_stages.py results/stress_s2`

`plot_crossover.py`, `plot_safe_comparison.py`, `plot_trust_comparison.py` EARLIER WORK

Figures from the first crossover and trust-formula experiments. The results they read have been superseded, so they are not used for the report.

Data preparation and SINCAL

`make_scenario_data.py` DATA PREPARATION

Made `data/pv_profile.csv` (solar) and `data/ambient.csv` (temperature). It has already run; rerun it only if the scenario changes.

Run: `py src/make_scenario_data.py`

`build_sincal_model.py` SINCAL

Writes the street into a SINCAL project by filling SINCAL's own database directly, because 906 buses is too many to draw. Used for the SINCAL cross-check.

`inspect_sincal_schema.py` SINCAL

Read out the layout of SINCAL's database so `build_sincal_model.py` could be written. Its output is `notes/sincal_schema.md`.

7. The checks (tests/)

The V checks each test a piece of the model against an answer that does not come from the simulator itself. They all pass at the moment: V2, V5, V6 and V7 against the stand-ins, until the teammates' code arrives.

`v1_benchmark.py` V1

The street model against the IEEE's published voltages: the worst node is 0.0002% out, against a limit of 0.5%. Writes `notes/v1_result.md`.

Run: `py tests/v1_benchmark.py`

`v3_envelope.py` V3

The safe-limit search finds the true answer: at 30 random points, compared with a slow step-by-step search, it is always within 0.1 kW.

Run: `py tests/v3_envelope.py`

`v4_tightness.py` V4

The safe limits are safe, and as large as they can be: applying one breaks nothing, and one step more would break something.

Run: `py tests/v4_tightness.py`

`v8_known_answer.py` V8

The whole chain against an answer worked out by hand: every house charging at full power, and simple arithmetic predicts the main cable first overloads at 16:40. The simulation agrees.

Run: `py tests/v8_known_answer.py`

`v9_reproducibility.py` V9

The same runs give identical logs a week later. It passed on 11 October. `--record` stores a new baseline, `--check` compares against it a week on.

Run: `py tests/v9_reproducibility.py --check`

`test_node_acceptance.py` V2, V5, V6, V7

The teammates' hand-over tests: V2 and V7 for Gia's battery (the AS/NZS 4777.2 response, energy balance), V5 and V6 for Hoang's comms link (the delay and loss asked for, sensible trust). They run on the stand-ins, or on a teammate's file with `--team`.

Run: `py tests/test_node_acceptance.py --team hoang`

`test_mqtt_equivalence.py` MQTT

Runs one evening in process and again through a real MQTT broker on this laptop, and requires every step to match. It is what makes the simulated results valid for the hardware.

Run: `py tests/test_mqtt_equivalence.py`

`test_setpoint_schedule.py` SCHEDULES

Checks that the setpoint-schedule fix (amendment 25) works as designed: the right entry at each step, the right margins, and nothing else disturbed.

Run: `py tests/test_setpoint_schedule.py`

`test_lomitha_adapter.py` LOMITHA

Checks Lomitha's controller as integrated. With his pilot rules and the car's taper switched off it must give the stand-in's answer exactly (it does, on five evenings); then it shows what the real rules change: 0.5 to 2.4% less energy, no extra violations.

Run: `py tests/test_lomitha_adapter.py`

`golden_runs.py` SAFETY NET

Reference runs stored in `results/golden/`. After any code change, `--check` reruns them and compares every field step by step, which proves the change did not alter a result.

Run: `py tests/golden_runs.py --check --flag off`

Run: `py tests/golden_runs.py --check --flag on`

`check_model.py` SINCAL

Checks a SINCAL database for broken references before SINCAL opens it.

`solve_timing.py` SPEED

Measures the seconds per power flow. It decided early on whether stage S1 could run on this laptop at all.

8. The helpers (tools/)

For the Raspberry Pi

`pi_session.py` YOU RUN THIS

One command for a whole run with the Pi. It rebuilds the Pi package, copies it over SSH, starts the broker here and the Pi's program there (through an SSH tunnel, so nothing needs allowing through the firewall), runs the evening, compares it with the simulator and stops the Pi's program. `--find` finds the Pi on a hotspot; `--local` runs everything on the laptop alone; `--battery-on-pi` puts the battery on the Pi too.

Run: `py tools/pi_session.py --find`

Run: `py tools/pi_session.py --pi <user>@<Pi address> --team all --battery-on-pi`

`run_with_pi.py` LAPTOP SIDE

The laptop side of a run with the Pi, which `pi_session.py` starts for you. It waits for the Pi to report in, runs an evening over MQTT showing every step, and with `--compare` checks the Pi gave the simulator's answer (EQUIVALENT), saying what differs if not. Each run is saved in `results/hil/`.

`pi/run_pi_node.py` THE PI SIDE

The program that runs on the Pi: it loads the cars for the seed, starts the charge-point controller (Lomitha's with `--team lomitha`) and answers the laptop; with `--battery` it runs the battery too (Gia's with `--team gia`). It waits until the broker has really accepted it before saying "connected", and stops by itself after the evening when started by `pi_session.py`.

`broker.py` PI NETWORK

An MQTT broker on the laptop, for when it is not running on the Pi. It prints the laptop's address to give the Pi.

Run: `py tools/broker.py`

`make_pi_package.py` PI PACKAGE

Builds `pi_package/` and `pi_package.zip`: the message contract, the charge-point controller (the stand-in and Lomitha's code), the battery (the stand-in and Gia's file), the MQTT wrapper, the runner, the cars for every seed, and a copy of the MQTT library, so the Pi needs nothing installed. `pi_session.py` rebuilds it on every run.

Run: `py tools/make_pi_package.py`

`make_handover.py` HANDOVER

Builds `EEE40016_for_Gia.zip`: the whole project for a teammate to run with a Pi, with `README_FIRST.md` on top. Leaves out logs, caches and superseded results.

Run: `py tools/make_handover.py`

For the experiments

`run_parallel.py` YOU RUN THIS

Runs many `stages.py` processes at once, 12 by default, each with its own log: for example all 13 seeds of S1. The final results were made with it.

Run: `py tools/run_parallel.py --arm "team:S1:results/stages_team:--team all"`

`hold_awake.py` HELPER

Stops Windows going to sleep while experiments are running, then lets go when they finish.

Run: `py tools/hold_awake.py`

`calibrate_preview.py` HELPER

A quick look at controller A's fixed limit for each seed in each demand case: a seed at 7.2 kW has no congestion, so live control has nothing to win there.

`demand_scenarios.py` HELPER

The screening that picked the demand cases (amendment 27): how much load unmanaged charging puts on the street, without any network model. Explained in `notes/demand_scenarios.md`.

`compare_paired.py, compare_sensitivity.py` HELPER

Scored the setpoint-schedule fix: the same runs with schedules off and on, against criteria fixed before they ran. Their reports are the `comparison.md` files in `results/paired_main/`, `paired_stress/` and `sensitivity/`.

For the SINCAL cross-check

`sincal_subset_check.py` SINCAL

SINCAL against OpenDSS on a 47-node cut of the street, small enough for the laptop's licence. They agree within 0.1%.

`make_avd_package.py, avd_solve.py, avd_compare.py` SINCAL

The full-street check on the uni computers (the AVD), which have the full licence. `make_avd_package.py` builds `avd_crosscheck.zip`, `avd_solve.py` solves it on the AVD, and `avd_compare.py` compares the answer with OpenDSS back on the laptop. Still to be done.

`sincal_repoint.py` SINCAL

Fixes a copied SINCAL project's link to its own database, which breaks when a project moves to another computer.

9. Teammates' files (team/)

Each teammate's part plugs in here. Hoang's and Gia's files still inherit the stand-ins, so they work before a line changes, and each replaces the methods with their own code; Lomitha's own code is already in.

`hoang_link.py` HOANG

`HoangLink` (the uplink and the trust score) and `HoangDownlink` (the downlink), still the stand-in. Must pass V5 and V6, and rules R6 and R7.

`hoang_prototype/` HOANG

Hoang's MQTT chat prototype (a publisher, a subscriber and a Mosquitto setting), with `NOTES.md` on how it fits: it is a connection test, not part of a run. Two small changes, listed in the notes.

`lomitha/` LOMITHA

Lomitha's own code: `ev_charge_point_gateway.py` (one charge point: the limit, the IEC 61851 pilot rules, metering, faults, the fallback) and `charging_session.py` (the car, with a taper above 80%, and Lomitha's own test runner). Two small changes, marked in the files.

`lomitha_fleet.py` LOMITHA

`LomithaFleet`, the adapter that runs one of Lomitha's gateways and cars per charge point, all 22 from one object, and keeps rules R1 to R5. Runs on the Pi; must give EQUIVALENT.

`gia_battery.py` GIA

`GiaBattery`, the battery, BMS and EMS, still the stand-in. Must pass V2 and V7; runs in the twin, or on the Pi with `--battery-on-pi`.

`README.md` COMMANDS

Every command for the integration session, ready to copy and paste.

`__init__.py`

An empty marker that lets Python load the files in this folder by name. Leave it there.

10. Results (results/)

Folder	What it holds
`stages/`	the main case (s1 demand): S1, S2, S3 and S1H for each seed, `summary.txt` and the three figures
`stress_s2/`	the stress case (s2 demand)
`paired_main/`, `paired_stress/`	the schedule fix: the same runs with schedules off, and the comparison
`mechanism_standin/`	the schedule fix tested on the older demand, before and after
`sensitivity/`	two variations of the schedule fix, and the comparison
`timeout_sensitivity/`	the timeout test: summary and figure
`golden/`	the reference runs for `golden_runs.py`
`v9/`	the V9 baseline
`hil/`	runs with the Raspberry Pi; `rehearsal/` holds the laptop rehearsals of 11 October
`sincal_subset_check.json`	the 47-node SINCAL comparison
`stages_team/`	appears after the final runs with everyone's code
`superseded/`	results from before a fault was fixed, or from an old scenario. Never quote these

Reading summary.txt

Each cell is one delay and loss level, and reads like `+6.5 13/13`: at that setting, controller B delivered 6.5% more energy than the fixed limit on average, and all 13 evenings broke no limit. `-1.6 12/13` means 1.6% less than the fixed limit, with one evening of the 13 breaking a limit; an evening that breaks a limit is left out of the average, as the plan says. The last lines count the runs with a violation and list the fixed limit found for each seed.

What one result holds

Each JSON file holds one record per run. The fields you will use most:

Field	Meaning
`controller`, `latency_s`, `loss`, `seed`	which run it was
`energy_kwh`	the main measure: EV energy delivered over the evening
`demanded_kwh`	the energy the cars wanted
`violations`	the number of steps that broke at least one limit
`violation_types`	which limits broke, and how often
`counted`	false if any limit broke; such a run does not count
`mean_trust`	the average trust score over the evening
`fallbacks`	the steps where the chargers fell back to 1.4 kW
`static_kw`, `static_kwh`	controller A's fixed limit for that seed, and the energy it delivers

11. Notes and documents

notes/

File	What it is
`AUDIT.md`	start here: every commitment in the research plan against the build, the results, and what is left
`plan_amendments.md`	all 28 design changes and faults found, each with the reason and the evidence
`demand_scenarios.md`	why s1 is the main case and s2 the stress case
`feeder_validation.md`	the checks on the IEEE feeder package
`v1_result.md`, `v3_result.md`, `v4_result.md`, `v8_result.md`	each check's result in detail
`first_crossover.md`	the first, preliminary result; superseded, kept for the record
`sincal_subset_check.md`, `sincal_schema.md`, `importer_open_items.md`, `BLOCKER_licence.md`	the SINCAL work: the 47-node check, SINCAL's database layout, importer issues, and the 50-node licence limit

docs/

File	What it is
`Digital_Twin_Guide.pdf`	the models, the method and the results, explained
`Putting_It_Together.pdf`	the Raspberry Pi set-up and the network
`Your_Files_Explained.pdf`	this guide
`INTERFACES.md`	the rules teammates' code must follow (R1 to R7), with example messages
`guide/`	the scripts that build the three PDFs, and `render_pages.ps1`, which turns PDF pages into pictures for checking

12. Everything else in the project folder

File or folder	What it is
`README_FIRST.md`	the steps for running everything with the Pi, for Gia; on top of the handover zip
`requirements_laptop.txt`	the Python packages the laptop side needs, one pip command
`README.md`	the project's front page: status, layout, decisions
`EEE40016_for_Gia.zip`	the handover zip, built by `tools/make_handover.py`
`pi_package/`, `pi_package.zip`	everything that goes on the Pi; built by `tools/make_pi_package.py`
`avd_crosscheck.zip`	the SINCAL check for the uni computers
`models/`	the SINCAL projects; the files starting with ~ are SINCAL's own backups
`logs/`	the logs from long runs, one per process
`LOG/`, `Log.db`	SINCAL's own log files